

模型实时化方案

一、问题背景

二、直播推荐的工业排序链路

三、Base 模型通常是什么

四、公开工业方案参考

1. 快手 Moment&Cross

2. 快手 OneLive

3. TikTok Live Zenith

五、根因分析

1. 稳态样本占绝大多数

2. 实时特征在离线指标中容易被低估

3. 标签存在延迟

六、作者方案总览

七、特征方案：构造实时特征

1. User 侧实时特征

2. Item 侧实时特征

八、样本方案：延迟回补

1. 原始问题

2. 新窗口设计

3. 回补梯度的数学逻辑

4. 工程实现方式

5. 简化版实现

九、模型方案：增加实时塔

1. 原模型结构

2. 新增实时塔

3. 为什么直接相加

4. 是否是 Aux

十、PCOC 和 ECE

1. PCOC

2. ECE

十一、RT 约束与 Gate 方案

十二、rt_logit 方差观察

十三、工业界可行的增强方案

1. 突变样本挖掘与加权

2. Real-time Delta 特征

3. 多窗口特征

4. 实时塔加轻量 Aux

5. 校准层

十四、推荐落地路径

第一阶段：诊断

第二阶段：特征增强

第三阶段：样本回补

第四阶段：实时塔

第五阶段：增强实时塔

十五、总结

参考资料

一、问题背景

在直播、短视频、推荐、广告等排序系统中，经常会遇到一个问题：线上对象的状态已经发生变化，但模型预估分没有及时反映出来。

以直播间排序为例，某个直播间在 t 时刻突然变好：

- 1 互动率升高
- 2 在线人数激增
- 3 评论、点赞、关注、送礼速率上涨
- 4 进房转化率变好
- 5 停留时长变好
- 6 负反馈下降

但模型排序分没有同步提升，导致这段时间内直播间排序分被低估，分配到的流量不足，在线人数进入平台期。

如果已经排除了流量池调控、冷启动扶持、运营策略等外部策略因素，那么问题大概率出在模型本身：

- 1 模型没有充分利用实时信号。

更具体地说，模型排序分主要由慢变特征驱动，例如：

- 1 用户长期画像
- 2 主播历史表现
- 3 直播间历史统计
- 4 用户长期兴趣
- 5 item 长期质量分

这些特征稳定、强势、容易学习，但它们无法捕捉秒级到分钟级的变化。实时特征虽然包含正确的短期信号，但在训练数据中占比小、噪声大、重要性低，模型容易忽略它们。

二、直播推荐的工业排序链路

直播推荐不是一个单模型问题，而是一个级联系统问题。一般链路如下：

本文讨论的模型实时化，主要发生在 **粗排**、**精排**、**重排** 阶段，尤其是精排 fullrank 模型。

三、Base 模型通常是什么

在工业直播推荐中，所谓 **base 模型** 通常不是一个简单二分类 DNN，而是精排 fullrank 多任务模型。

它输入：

- 1 用户画像
- 2 用户短期/长期行为序列
- 3 主播/直播间历史特征
- 4 直播间实时状态
- 5 上下文特征
- 6 内容 embedding / 多模态 embedding

经过：

- 1 Embedding
- 2 特征交叉
- 3 用户序列兴趣建模
- 4 多任务学习模块

输出多个目标概率：

- 1 pCTR
- 2 pLongView
- 3 pLike
- 4 pComment
- 5 pFollow
- 6 pGift
- 7 pWatchTime

线上再用业务公式融合成排序分。

所以可以把 base 模型理解成：

- 1 多特征 embedding + 特征交互主干 + 用户兴趣序列模块 + 多任务塔

常见组件包括：

- 1 Embedding：稀疏 ID、类目、统计特征离散化
- 2 DNN：基础非线性表达
- 3 DCN / DeepFM / CrossNet：显式特征交叉
- 4 DIN / DIEN / Target Attention：用户历史兴趣抽取
- 5 MMoE / PLE：多目标学习
- 6 Calibration：校准 pCTR / pCVR / pXTR

四、公开工业方案参考

截至 2026-04-29，公开资料里和直播间排序/实时化最相关的方案主要有：

公司/场景	公开方案	定位	关键词
快手直播	Moment&Cross, 2024	实时 CTR/fullrank 排序	30s 实时样本流、first-only mask、短视频到直播兴趣迁移、MMoE/PLE
快手直播	LiveForesighter, 2025	预测未来直播状态	生成未来信息、当前时刻和未来一段时间推荐
快手直播	OneLive, 2026	生成式直播推荐框架	Dynamic Tokenizer、Time-Aware Gated Attention、Decoder-only、多目标对齐

快手直播	KuaiLive, 2025/2026	直播推荐数据集	click/comment/like/gift、实时交互、top-K/CTR/watch/gift 任务
TikTok Live/字节	Zenith, 2026	大规模直播排序模型	Prime Tokens、Token Fusion、Token Boost、低延迟扩容
抖音/抖音 Lite	Streaming VQ, 2025	实时召回索引	实时给 item 挂索引、替换主要 retriever、提升召回实时性

1. 快手 Moment&Cross

Moment&Cross 的问题非常贴近本文：直播间内容是动态变化的，同一个主播在聊天、才艺展示、PK、游戏关键局等不同阶段，用户点击率和互动率会明显不同。

它想解决的问题是：

- 1 如何把直播间在正确的时刻推荐给用户？

它的核心做法有两个。

第一，**Moment**：把训练样本流从原来的 5 分钟/1 小时快慢流，升级为 30 秒实时上报，让模型更快感知直播间 high-light moment。

第二，**Cross**：利用用户大量短视频行为迁移到直播推荐，缓解直播曝光少、数据稀疏的问题。

它和本文提到的 **ymin + zmin 回补** 属于同一类思想：

- 1 不要让直播长反馈把模型训练拖慢
2 也不要把延迟行为误杀成负样本

Moment&Cross 里还提到直播 fullrank 模型通常是多任务二分类范式，多任务模块可以用 MMoE / PLE。这也说明工业直播 base 模型往往不是单目标 CTR 模型，而是多目标 XTR 模型。

2. 快手 OneLive

OneLive 是更前沿的生成式直播推荐框架。它不是只优化精排，而是试图把传统级联推荐统一成生成式推荐。

核心组件包括：

- 1 Dynamic Tokenizer: 动态编码实时直播内容
- 2 Time-Aware Gated Attention: 建模时间变化
- 3 Decoder-only Generative Architecture: 生成式推荐
- 4 Sequential MTP + QK Norm: 稳定训练和加速推理
- 5 Multi-Objective Alignment: 多目标对齐

这类方案工程门槛很高，不适合作为第一版实时化方案。更现实的路线仍然是：

- 1 先在现有 fullrank base 模型上加实时特征、实时样本流、实时塔。

3. TikTok Live Zenith

Zenith 更像是在回答另一个问题：

- 1 直播排序模型能不能在低延迟约束下继续扩大模型容量？

它的思路不是简单堆大 MLP，而是把特征组织成少量高维 Prime Tokens，再做 Token Fusion 和 Token Boost，让模型能学复杂交互，同时控制线上推理开销。

这说明新的大规模排序模型正在往 token 化特征交互、可扩容、低延迟 方向演进。

五、根因分析

1. 稳态样本占绝大多数

大部分训练样本来自直播间稳定状态：

- 1 实时互动率 $\approx$ 历史互动率
- 2 实时在线人数趋势 $\approx$ 历史趋势
- 3 实时转化率 $\approx$ 长期转化率

在这种情况下，离线特征已经足够解释标签，模型自然会优先依赖离线特征。

真正有价值的突变样本反而很少：

- 1 直播间突然爆发
- 2 主播突然进入高互动状态
- 3 用户兴趣在 session 内突然转移
- 4 短时间内流量质量变化

这些样本在整体训练集中占比低，模型很难充分学习。

2. 实时特征在离线指标中容易被低估

实时特征的价值主要体现在：

- 1 更快响应线上变化
- 2 减少排序滞后
- 3 改善短时间流量分配
- 4 修正在线人数平台期
- 5 提升互动类指标

但普通离线 AUC 通常是在整体样本上算的，稳态样本占比太高，所以实时特征的收益不一定明显。

因此，实时化项目不能只看整体 AUC，还要关注：

- 1 突变样本 AUC
- 2 GAUC
- 3 分桶 PCOC
- 4 ECE
- 5 延迟窗口 LogLoss
- 6 在线互动指标
- 7 在线人数曲线

3. 标签存在延迟

很多行为不是曝光后立刻发生的。

比如直播场景：

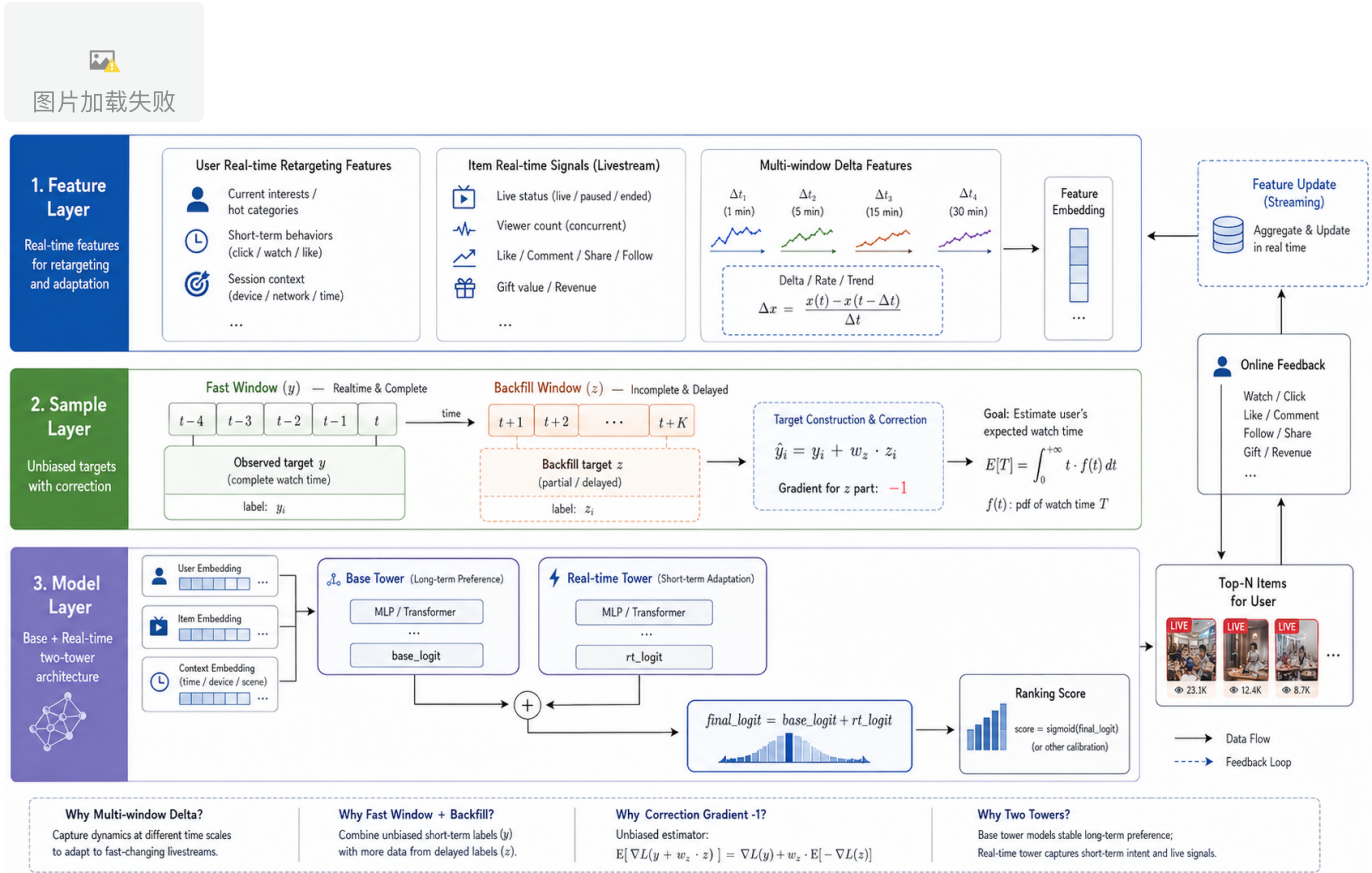
- 1 用户曝光直播间
- 2 几秒后进房
- 3 几十秒后互动
- 4 几分钟后关注或送礼

如果训练样本只用一个较短窗口，例如 `x min`，那么超过窗口才发生行为的样本会被当成负样本。

这会产生系统性问题：

- 1 本来是正样本，但先被当成负样本训练
- 2 模型学习到错误梯度
- 3 模型对延迟正样本系统性低估

六、作者方案总览



读图说明： 这张图对应本文的总方案。左侧是特征层，包括 user 侧实时重定向特征和 item 侧短窗口实时信号；中间是样本层，通过 y 快速窗口和 z 延迟回补窗口修正迟到正样本；右侧是模型层，通过 **Base Tower + Realtime Tower** 让实时信号直接进入最终排序分。图中的 **correction gradient = -1** 对应回补样本的补偿梯度，**final_logit = base_logit + rt_logit** 对应实时塔在 logit 空间对 base 分做修正。

作者方案可以概括成三部分：

- 1 特征实时化
- 2 样本实时化
- 3 模型结构实时化

分别对应三个问题：

- 1 模型看不到实时变化 => 构造实时特征
- 2 模型学不到延迟正样本 => 做样本回补
- 3 模型即使看到也用不强 => 增加实时塔

七、特征方案：构造实时特征

1. User 侧实时特征

User 侧主要做实时兴趣捕捉，也可以理解为重定向特征。

目标是：尽量把用户推给他当前更可能喜欢的直播间。

常见特征包括：

- 1 用户最近 5 秒 / 30 秒 / 1 分钟点击过的直播间类型
- 2 用户最近一次进房的主播类目
- 3 用户当前 session 内互动过的直播间标签
- 4 用户短期停留时长偏好的主播类型
- 5 用户最近是否对某类内容负反馈减少
- 6 用户当前是否从泛兴趣转向强兴趣

长期画像告诉模型：

- 1 这个用户平时喜欢什么

实时特征告诉模型：

- 1 这个用户现在正在看什么、点什么、对什么感兴趣

2. Item 侧实时特征

Item 侧是模型实时化的重点。它要捕捉直播间短期状态变化，例如：

- 1 1 分钟互动率
- 2 30 秒互动率
- 3 5 分钟互动率
- 4 在线人数增长率
- 5 进房转化率
- 6 评论速率
- 7 点赞速率
- 8 关注速率
- 9 送礼速率
- 10 人均停留时长变化
- 11 负反馈率变化
- 12 实时退出率
- 13 短窗口成交率

还可以构造趋势类特征：

- 1 当前 1 分钟互动率 / 过去 10 分钟互动率
- 2 当前在线人数 / 过去 5 分钟平均在线人数
- 3 当前评论速率 - 历史评论速率
- 4 当前进房转化率的环比增长
- 5 短窗口质量分相对长期质量分的偏差

实时化里最有价值的往往不是：

- 1 这个直播间互动率高

而是：

- 1 这个直播间现在比它平时更好

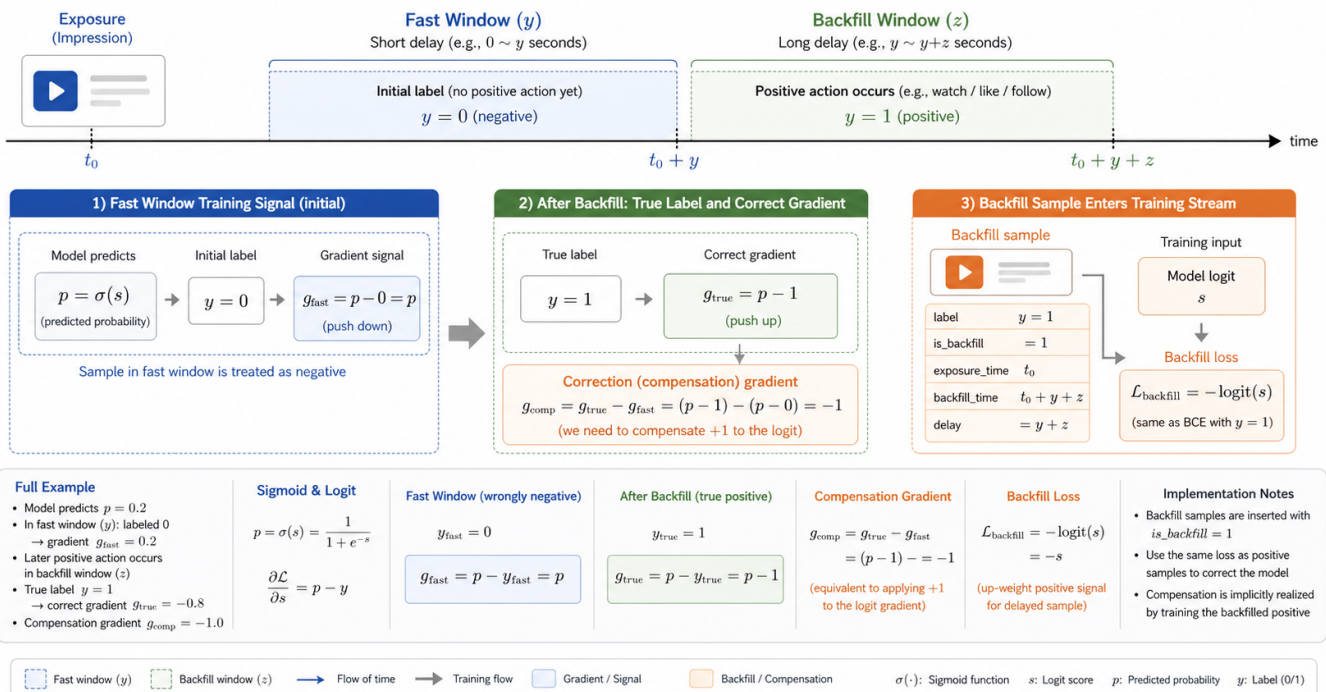
所以 delta 特征非常关键。

八、样本方案：延迟回补



图片加载失败

Delayed Positive Backfill and Correction Gradient for Real-time Ranking



读图说明： 这张图从时间线解释回补机制。曝光发生在 t_0 ，短窗口 y 内暂时没有行为时，样本先被当作负样本训练，已使用梯度是 $p - 0 = p$ 。后续在回补窗口 z 内发生正行为，正确梯度应该

是 $p - 1$ ，所以补偿梯度为 $(p - 1) - (p - 0) = -1$ 。工程上可以用 `loss_backfill = -logit` 让回补样本对 logit 的梯度固定为 `-1`。

1. 原始问题

假设原来的样本窗口是 `x min`：

- 1 曝光后 `x` 分钟内发生行为 => 正样本
- 2 曝光后 `x` 分钟内没有行为 => 负样本

问题是，有些真正正行为发生在 `x min` 之后。

例如：

- 1 曝光后 20 秒内没有互动，被标为负样本
- 2 曝光后 2 分钟发生关注

旧版本会把它当负样本训练，这会导致模型低估这类样本。

2. 新窗口设计

作者把窗口改成类似：

- 1 `y min + z min`

可以理解为：

- 1 `y min`：快速窗口
- 2 `z min`：延迟回补窗口

快速窗口负责让短期正样本尽快进入训练：

- 1 曝光后很快发生行为 => 立即作为正样本训练

延迟窗口负责修正迟到正样本：

- 1 短窗口内暂时没行为，先按负样本训练
- 2 后续窗口内发现行为，再做回补修正

3. 回补梯度的数学逻辑

二分类 BCE 在 logit 上的梯度是：

```
1  p - y
```

其中：

```
1  p = sigmoid(logit)
2  y = label
```

如果一个样本一开始被当作负样本：

```
1  y = 0
2  梯度 = p - 0 = p
```

后来发现它其实是正样本：

```
1  y = 1
2  正确梯度 = p - 1
```

那么需要补偿的梯度是：

```
1  补偿梯度 = 正确梯度 - 已经使用过的梯度
2           = (p - 1) - (p - 0)
3           = -1
```

所以回补时不再用普通正样本 loss，而是让这个样本对 logit 的梯度固定为 `-1`。

4. 工程实现方式

如果使用 PyTorch，可以通过构造特殊 loss 实现。

因为：

```
1  loss = -logit
2  d(loss) / d(logit) = -1
```

所以回补样本可以这样写：

```

1  logit = model(features)
2
3  bce_loss = F.binary_cross_entropy_with_logits(
4      logit,
5      label.float(),
6      reduction="none"
7  )
8
9  backfill_loss = -logit
10
11 loss = torch.where(
12     is_backfill.bool(),
13     backfill_loss,
14     bce_loss
15 )
16
17 loss = (loss * sample_weight).mean()

```

注意，固定为 `-1` 的是对 `logit` 的梯度，不是对所有模型参数的梯度。模型参数仍然通过链式法则更新：

```

1  dL/dw = dL/dlogit * dlogit/dw

```

5. 简化版实现

如果训练框架不支持自定义 `loss`，可以先做工程简化：

- 1 延迟窗口发现的正样本，重新作为正样本训练一次
- 2 并设置较大的 `sample_weight`

例如：

- 1 普通正样本 `weight = 1`
- 2 普通负样本 `weight = 1`
- 3 回补正样本 `weight = 2 ~ 5`

这种方式不如补偿梯度精确，但更容易落地。

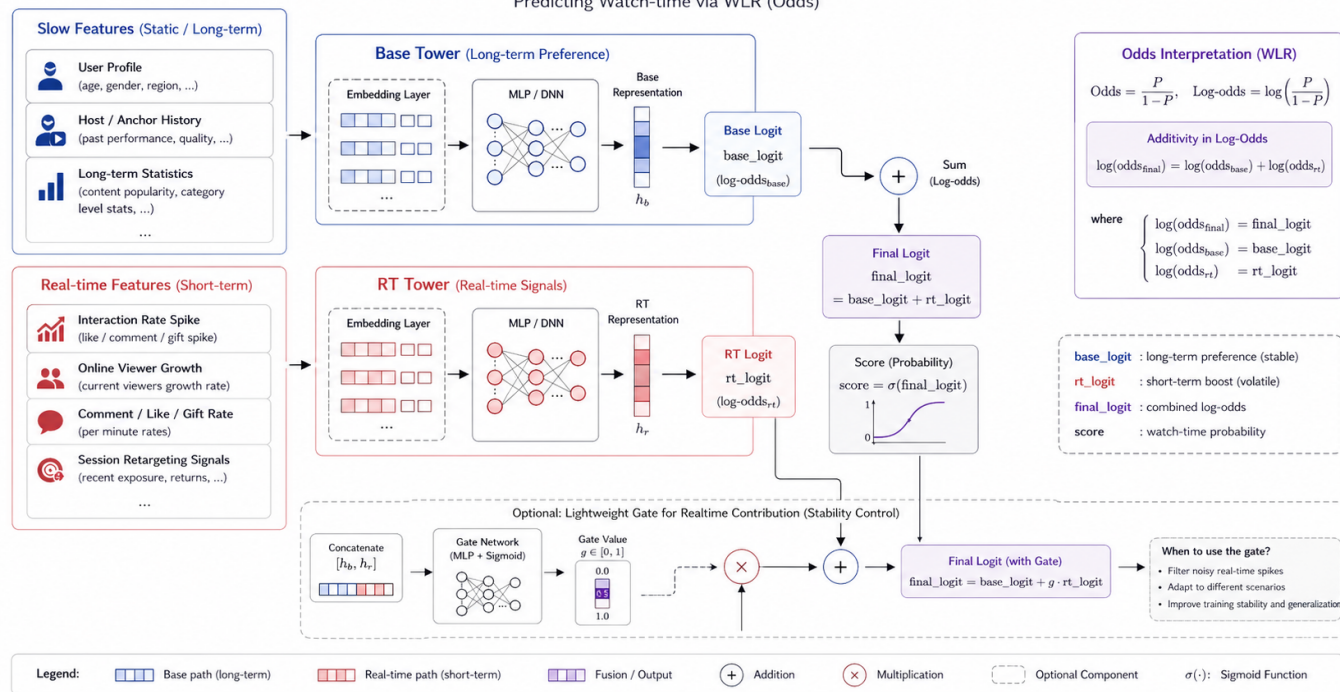
九、模型方案：增加实时塔



图片加载失败

Real-time Ranking Model: Base Tower + RT Tower

Predicting Watch-time via WLR (Odds)



读图说明：图中上路是慢变特征进入 Base Tower，输出 base_logit；下路是实时特征进入 RT Tower，输出 rt_logit。二者在 logit 空间相加：final_logit = base_logit + rt_logit。右侧给出 odds 解释：log(odds_final)=log(odds_base)+log(odds_rt)，所以两个 logit 相加等价于两个 odds 相乘。虚线框中的 gate 是可选增强，用于实时信号质量不稳定时控制实时塔贡献。

1. 原模型结构

原模型可能是一个 base tower:

```
1 base_logit = BaseTower(user features, item features, context features)
2 score = sigmoid(base_logit)
```

base tower 输入通常包含：

- 1 用户画像
- 2 用户长期兴趣
- 3 主播历史统计
- 4 直播间历史质量
- 5 上下文特征
- 6 部分实时特征

但由于实时特征弱、稀疏、噪声大，模型可能没有充分利用它们。

2. 新增实时塔

新增实时塔：

```
1 rt_logit = RealtimeTower(realtime features)
```

最终输出：

```
1 final_logit = base_logit + rt_logit
2 score = sigmoid(final_logit)
```

这个方案的好处是：

- 1 base 模型继续负责长期匹配、用户偏好、主播历史质量
- 2 实时塔专门负责直播间当前状态的短期修正
- 3 两个 logit 相加，在 odds 空间形成乘性修正

3. 为什么直接相加

备注：logit就是概率 p 经过计算赔率转对数得到的正负无穷的可线性建模的实数，这个实数再还原成概率 p 就是sigmoid激活函数

logit 是 $\log \text{ odds}$ ：

```
1 logit(p) = log(p / (1 - p))
```

其中：赔率，发生/不发生

```
1 odds = p / (1 - p)
```

如果：

```
1 base_logit = log(odds_base)
2 rt_logit = log(odds_rt)
```

那么：

```
1 final_logit = base_logit + rt_logit
2             = log(odds_base) + log(odds_rt)
3             = log(odds_base * odds_rt)
```

也就是说，两个 logit 相加，等价于两个 odds 相乘。

直观理解：

- 1 base tower 判断长期匹配程度
- 2 realtime tower 判断当前实时状态
- 3 最终排序分 = 长期匹配 × 实时状态修正

这就是“在概率赔率空间里对 base 结果做乘性修正”。

4. 是否是 Aux

这个实时塔有点像 aux，但不完全是 aux。

更准确地说，它是一个实时残差塔：

- 1 final_logit = base_logit + rt_logit
- 2 loss = BCE(final_logit, label)

通常只对最终输出算主 loss。

它和普通 aux loss 的区别是：

- 1 实时塔：结构上强化实时特征通路
- 2 Aux：额外引入辅助监督任务

普通 aux 可能是：

- 1 主任务：点击
- 2 辅助任务：停留
- 3 辅助任务：互动
- 4 辅助任务：关注

loss 写成：

- 1 loss = click_loss + α * stay_loss + β * interact_loss

如果要增强实时塔，也可以增加轻量 aux：

- 1 main_loss = BCE(final_logit, main_label)
- 2 rt_aux_loss = BCE(rt_logit, short_window_label)
- 3 loss = main_loss + α * rt_aux_loss

其中 `short_window_label` 可以是短窗口点击、短窗口互动、短窗口进房、短窗口强反馈等。

十、PCOC 和 ECE

1. PCOC

PCOC 常见定义是：

$$1 \quad PCOC = \text{sum}(\text{prediction}) / \text{sum}(\text{label})$$

如果是 CTR 场景：

$$1 \quad PCOC = \text{预测点击数} / \text{实际点击数}$$

例子：

- 1 10000 次曝光
- 2 真实点击 100 次
- 3 模型预测概率总和 80
- 4 $PCOC = 80 / 100 = 0.8$

说明模型整体低估。

解释：

- 1 $PCOC = 1$ ：校准准确
- 2 $PCOC > 1$ ：模型高估
- 3 $PCOC < 1$ ：模型低估

PCOC 常用于：

- 1 广告 CTR/CVR 预估
- 2 推荐点击率预估
- 3 转化率预估
- 4 直播互动率预估
- 5 金融风险概率预估

2. ECE

ECE 是 Expected Calibration Error，期望校准误差。

做法：

- 1 按预测概率分桶
- 2 每个桶计算平均预测概率 `avg_pred`
- 3 每个桶计算真正例率 `avg_label`
- 4 计算二者差值的加权平均

公式：

$$1 \quad ECE = \sum (n_b / N) * |avg_pred_b - avg_label_b|$$

其中：

- 1 `n_b` = 第 `b` 个桶的样本数
- 2 `N` = 总样本数
- 3 `avg_pred_b` = 该桶平均预测概率
- 4 `avg_label_b` = 该桶真正例率

AUC 关注排序能力：

- 1 正样本是否排在负样本前面

ECE/PCOC 关注校准能力：

- 1 模型说 10% 的时候，真实是否接近 10%

实时化场景下，PCOC 和 ECE 很重要，因为问题往往不是模型完全排错，而是：

- 1 模型系统性低估了某些正在变好的样本

十一、RT 约束与 Gate 方案

作者说没有用复杂 gate，因为线上 RT 约束比较紧。

这里 RT 通常指：

- 1 Response Time

也就是线上请求响应耗时。

推荐排序链路通常包括：

- 1 召回
- 2 粗排
- 3 精排
- 4 重排
- 5 策略
- 6 特征读取
- 7 模型推理
- 8 结果返回

每一步都有延迟预算。直播推荐尤其看重实时性，如果模型推理多几毫秒，可能会影响整体链路稳定性。

如果做 gate，可以先做轻量版本：

```
1 gate = sigmoid(W * gate_features + b)
2 final_logit = base_logit + gate * rt_logit
```

或者：

```
1 gate = 0.1 + 0.9 * sigmoid(...)
```

给 gate 设置下限，避免实时塔被完全关闭。

Gate 可以利用的信息包括：

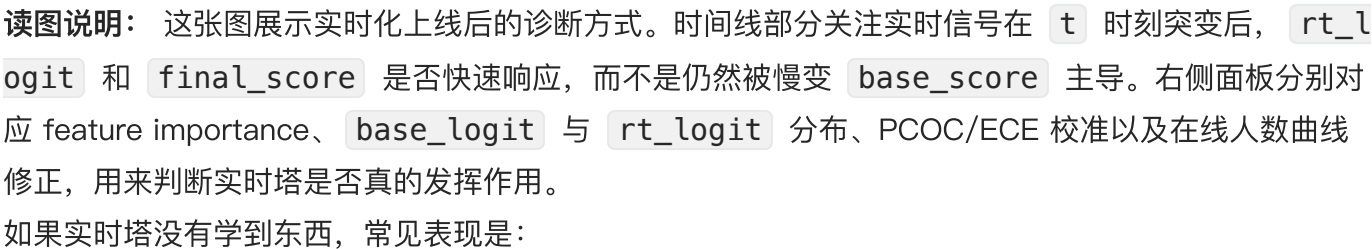
- 1 实时窗口样本量
- 2 短窗口统计是否稳定
- 3 直播间当前流量规模
- 4 当前是否冷启动
- 5 用户是否有强短期兴趣
- 6 base 分是否已经很高
- 7 实时特征缺失率
- 8 实时信号波动幅度

但 gate 有风险：它可能学成长期接近 0。因为 base 特征强，模型发现不用实时塔也能在大多数稳态样本上取得不错 loss，于是 gate 会压制实时塔。

解决办法包括：

- 1 给 gate 设置下限
- 2 增加突变样本权重
- 3 对实时塔加 aux loss
- 4 对实时样本 oversampling
- 5 监控 gate 分布
- 6 监控 rt_logit 分布

 图片加载失败



- 这说明实时塔基本没有影响最终排序。

但方差不小只说明“有变化”，不代表“变化正确”。还需要看：

- 1 rt_logit 是否和实时互动率正相关
- 2 rt_logit 高的样本真实表现是否更好
- 3 突变样本中 rt_logit 是否及时抬升
- 4 加入 rt_logit 后 PCOC/ECE 是否改善
- 5 线上指标是否提升

模型 forward 时可以输出中间结果：

```
1 class Model(nn.Module):
2     def forward(self, features, return_parts=False):
3         base_logit = self.base_tower(features)
4         rt_logit = self.rt_tower(features["rt_features"])
5         final_logit = base_logit + rt_logit
6
7         if return_parts:
8             return {
9                 "base_logit": base_logit,
10                "rt_logit": rt_logit,
11                "final_logit": final_logit,
12            }
13
14         return final_logit
```

离线评估时统计：

```
1 with torch.no_grad():
2     outputs = model(batch, return_parts=True)
3
4     base_logit = outputs["base_logit"]
5     rt_logit = outputs["rt_logit"]
6     final_logit = outputs["final_logit"]
7
8     print("base mean:", base_logit.mean().item())
9     print("base std:", base_logit.std().item())
10    print("rt mean:", rt_logit.mean().item())
11    print("rt std:", rt_logit.std().item())
12    print("final std:", final_logit.std().item())
```

更推荐写到分析表：

```
1 room_id
2 user_id
3 timestamp
4 base_logit
5 rt_logit
6 final_logit
7 base_score
8 final_score
9 rt_interact_rate_1m
10 rt_online_growth_1m
11 label
```

然后按直播间画时间序列：

```
1 实时互动率是否上升
2 rt_logit 是否上升
3 final_score 是否上升
4 在线人数是否恢复增长
```

十三、工业界可行的增强方案

1. 突变样本挖掘与加权

既然问题来自突变样本占比低，就应该显式构造突变样本集。

可以定义突变样本：

```
1 当前 1 分钟互动率 / 过去 10 分钟互动率 > threshold
2 当前在线人数增长率 > threshold
3 短窗口转化率明显高于长期转化率
4 实时质量分大幅偏离历史质量分
```

训练时提高这类样本权重：

```
1 normal sample weight = 1
2 burst sample weight = 2 ~ 5
3 delayed positive weight = 2 ~ 5
```

2. Real-time Delta 特征

不要只喂实时绝对值，还要喂相对变化。

例如：

```
1  rt_ctr_1m
2  hist_ctr_1d
3  rt_ctr_1m / hist_ctr_1d
4  rt_ctr_1m - hist_ctr_1d
5  rt_ctr_1m / rt_ctr_10m
6  rt_online_cnt_slope
```

3. 多窗口特征

实时信号容易噪声大，只看一个窗口不稳定。

建议同时使用：

```
1  10 秒窗口
2  30 秒窗口
3  1 分钟窗口
4  3 分钟窗口
5  5 分钟窗口
```

短窗口反应快，但噪声大；长窗口稳定，但反应慢。多窗口能平衡响应速度和稳定性。

4. 实时塔加轻量 Aux

如果条件允许，可以增加一个轻量 aux：

```
1  rt_logit = realtime_tower(rt_features)
2  final_logit = base_logit + rt_logit
3
4  main_loss = BCE(final_logit, main_label)
5  rt_aux_loss = BCE(rt_logit, short_window_label)
6
7  loss = main_loss +  $\alpha$  * rt_aux_loss
```

其中 short_window_label 可以是：

```
1  短窗口是否互动
2  短窗口是否进房
3  短窗口是否停留超过阈值
4  短窗口是否发生强反馈
```

aux 权重要小心，例如：

```
1  $\alpha = 0.05 \sim 0.2$ 
```

并且要观察是否和主目标冲突。

5. 校准层

实时塔加入后，模型可能出现局部高估或低估。

可以增加校准层：

```
1 final_logit_calibrated = a * final_logit + b
```

或者按场景做分桶校准：

- 1 按直播间规模
- 2 按主播类型
- 3 按用户活跃度
- 4 按实时互动率分桶
- 5 按冷启动状态

重点观察：

- 1 PCOC
- 2 ECE
- 3 分桶 PCOC

十四、推荐落地路径

第一阶段：诊断

先确认问题是否真实存在：

- 1 实时互动率上升
- 2 模型分没有上升
- 3 流量没有及时增加
- 4 在线人数进入平台期

输出分析表：


```
1 room_id
2 timestamp
3 base_score
4 final_score
5 rt_features
6 label
7 traffic
8 online_cnt
9 interact_rate
```

观察分数和实时信号之间的延迟。

第二阶段：特征增强

先不上复杂模型，补实时特征：

```
1 item 多窗口实时特征
2 item delta 特征
3 user session 实时兴趣特征
4 实时特征缺失标记
5 实时窗口样本量特征
```

验证：

```
1 GAUC
2 突变样本 AUC
3 分桶 PCOC
4 在线小流量实验
```

第三阶段：样本回补

引入：

```
1 快速窗口 y
2 延迟窗口 z
3 is_backfill 标记
4 回补样本特殊 loss
```

如果框架不支持自定义 loss，先用加权正样本近似。

重点看：

- 1 延迟正样本召回
- 2 PCOC
- 3 ECE
- 4 延迟窗口 LogLoss
- 5 在线互动指标

第四阶段：实时塔

增加：

- 1 base_logit
- 2 rt_logit
- 3 final_logit = base_logit + rt_logit

先不加 gate，不加复杂 aux。

观察：

- 1 rt_logit std
- 2 rt_logit 分布
- 3 rt_logit 与实时特征关系
- 4 rt_logit 对 final_score 的贡献

第五阶段：增强实时塔

如果实时塔有效，再考虑：

- 1 轻量 gate
- 2 rt aux loss
- 3 突变样本加权
- 4 校准层

不要一开始就把结构做复杂，否则很难定位收益来自哪里。

十五、总结

模型实时化不是简单加几个实时特征，而是一个系统工程。

作者方案可以总结为：

- 1 特征层：构造 user/item 实时特征，让模型看到实时变化
- 2 样本层：做延迟回补，修正迟到正样本被误判为负样本的问题
- 3 模型层：增加实时塔，让实时信号直接进入 final logit

结合公开工业方案看，快手 Moment&Cross 证明了 30 秒实时样本流、first-only mask、跨域兴趣迁移对直播推荐很重要；TikTok Live Zenith 说明大规模直播排序模型正在往 token 化、低延迟、可扩容特征交互方向演进；OneLive 则代表更前沿的生成式直播推荐方向。

更现实的工业落地路线是：

- 1 先诊断分数滞后
- 2 再补实时特征
- 3 再做样本回补
- 4 再加实时塔
- 5 最后考虑 gate、aux、校准

其中最关键的判断是：**实时化项目不能只看整体离线 AUC。**

更应该看：

- 1 突变样本表现
- 2 PCOC
- 3 ECE
- 4 分桶 LogLoss
- 5 rt_logit 分布
- 6 实时信号变化到模型分变化的延迟
- 7 在线互动指标
- 8 在线人数曲线

一句话概括：

- 1 模型实时化的本质，是让模型从“看历史做判断”，升级为“结合历史与当前状态做判断”。

参考资料

- [Moment&Cross: Next-Generation Real-Time Cross-Domain CTR Prediction for Live-Streaming Recommendation at Kuaishou](#)
- [LiveForesighter: Generating Future Information for Live-Streaming Recommendations at Kuaishou](#)
- [KuaiLive: A Real-time Interactive Dataset for Live Streaming Recommendation](#)
- [Zenith: Scaling up Ranking Models for Billion-scale Livestreaming Recommendation](#)

- [OneLive: Dynamically Unified Generative Framework for Live-Streaming Recommendation](#)
- [Real-time Indexing for Large-scale Recommendation by Streaming Vector Quantization Retriever](#)